4 Árvores e Florestas

Chamamos de ***acíclico*** qualquer grafo que não contenha circuito. Uma ***floresta*** é um grafo acíclico e os componentes conexos de uma floresta são árvores, ou seja, uma ***árvore*** é um grafo conexo e acíclico.

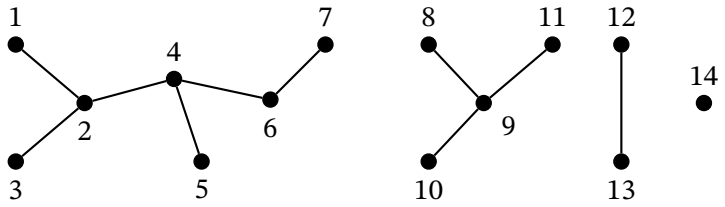


Figura 4.1: Uma floresta com 4 árvores.

Notemos que todo grafo acíclico G com pelo menos dois vértices tem pelo menos dois vértices de grau 1, basta tomarmos os extremos de um caminho de comprimento máximo (veja a Figura 1.6, pág. 5).

Chamamos de ***folha*** todo vértice de grau 1 em uma árvore.

Proposição 4.1

Toda árvore com n vértices tem $n - 1$ arestas.

Demonstração. Vamos provar por indução em $n \geq 1$ que a seguinte afirmação vale: *se um grafo com n vértices é uma árvore então o número de arestas é $n - 1$.*

Se o grafo é trivial, isto é $n = 1$, então o número de arestas é 0. Para $n \geq 2$, vamos assumir que toda árvore com $n - 1$ vértices tem $n - 2$ arestas e seja G uma árvore com n vértices.

Tome $v \in V(G)$ uma folha de G. O grafo $G - v$, dado por

$\nabla_G(U)$, o corte definido por U, foi definido no Exercício 60

$$G - v \coloneqq \big(V(G) \setminus \{v\},\, E \setminus \nabla_G(\{v\})\big)$$

é uma árvore (justifique) com $n - 1$ vértices e pela hipótese indutiva $|E(G - v)| = n - 2$. Como $|E(v)| = 1$ e $|E(G - v)| = |E(G) \setminus E(v)|$, temos que $|E(G)| = n - 1$. Portanto, pelo Princípio de Indução Finita, a afirmação vale para todo $n \geq 1$. □

Por outro lado, grafos conexos com uma aresta a menos que o número de vértices são árvores. Proveremos esse fato mais adiante.

Definição 4.1: Remoção e adição de aresta

Se $G = (V, E)$ é um grafo e e uma aresta de G, então definimos o grafo obtido pela remoção de e, denotado por $G - e$, por

$$G - e \coloneqq \big(V, E \setminus \{e\}\big).$$

Sejam $x, y \in V(G)$ vértices distintos de G. Definimos o grafo $G + xy$ por

$$G + xy \coloneqq \Big(V(G), E(G) \cup \big\{\{x, y\}\big\}\Big).$$

Note que, sendo x e y adjacentes, $G + xy = G$.

Teorema 4.1: caracterização de árvore

As seguintes afirmações são equivalentes para todo grafo $G = (V, E)$:

(1) G é árvore;
(2) para quaisquer $x, y \in V$ existe um único caminho em G com extremos x e y;
(3) G é *conexo minimal*: G é conexo e $G - e$ é desconexo, para qualquer $e \in E$;
(4) G é *acíclico maximal*: G é acíclico e $G + xy$ contém circuito, para quaisquer $x, y \in V$ não adjacentes em G.

Demonstração. Para G trivial o teorema vale como pode-se verificar facilmente. Se G tem ordem 2, então $G = K^2$ e a verificação também é fácil, deixa-mo-lá para o leitor. Vamos supor que G tem pelo menos 3 vértices e mostraremos (1)⇒(2)⇒(3)⇒(4)⇒(1).

(1) ⇒ (2): Seja G uma árvore e sejam $x, y \in V$. Se $x = y$, o único caminho é o trivial. Suponha $x \neq y$. Como G é conexo, existe um caminho de x a y. Suponha, por absurdo, que $P = x_0, x_1, \dots, x_{n+1}$ e $Q = y_0, y_1, \dots, y_{m+1}$ sejam caminhos distintos com $x_0 = y_0 = x$ e $x_{n+1} = y_{m+1} = y$. Seja

$$p \coloneqq \max\{i \geq 0 : \ x_k = y_k \text{ para todo } 0 \leq k \leq i\}.$$

Como y ocorre em Q apenas na última posição (e em P também), $p = n+1$ ou $p = m+1$ forçaria $n = m$ e $P = Q$. Logo $p \leq \min\{n, m\}$, os vértices x_{p+1} e y_{p+1} existem e $x_{p+1} \neq y_{p+1}$. Seja

$$q \coloneqq \min\{j : \ j > p \text{ e } x_j = y_\ell \text{ para algum } \ell > p\},$$

e fixe $\ell > p$ com $x_q = y_\ell$. Este q existe, pois $j = n + 1$ serve (com $\ell = m + 1$). Considere

$$C = x_p, x_{p+1}, \dots, x_q = y_\ell, y_{\ell-1}, \dots, y_{p+1}, y_p = x_p.$$

Seus vértices são distintos: P e Q são caminhos, e, pela minimalidade de q, nenhum x_j com $p < j < q$ aparece em $y_{p+1}, \dots, y_m$. Além disso, C tem $(q - p) + (\ell - p)$ arestas, com ambas as parcelas ≥ 1 e não ambas iguais a 1 (senão $x_{p+1} = y_{p+1}$); logo C tem comprimento ≥ 3 e é um circuito em G, contradição.

(2) ⇒ (3): Seja G tal que (2) vale. Por hipótese G é conexo. Para toda aresta $e = \{u, v\} \in E$ temos que $P = u, v$ é o único caminho com extremos u e v, portanto, $G - e$ é desconexo.

(3) ⇒ (4): Seja G conexo minimal. Se G contém circuito, então para qualquer $e \in E(G)$ que pertença a um circuito temos que $G - e$ é conexo (Proposição 1.3), uma contradição. Agora, sejam $x, y \in V$ distintos e não adjacentes em G; como G é conexo existe um caminho, digamos $P = x, v_1, \dots, v_k, y$, com extremos x e y. Em $G + xy$ temos o circuito $x, v_1, \dots, v_k, y, x$ pois $k \geq 1$.

(4) ⇒ (1): Seja G acíclico maximal; resta provar que G é conexo. Suponha que não: existem $x \neq y$ sem caminho entre eles em G. Em particular $xy \notin E(G)$, e por (4) o grafo $G + xy$ tem um circuito C. Como G é acíclico, $xy \in E(C)$, e então $C - xy$ é um caminho de x a y em G, contradição. □

Como consequência do Teorema 4.1 podemos concluir que *uma árvore com n vértices é um grafo conexo com o menor número possível de arestas dentre todos os grafos conexos com n vértices.*

Subárvores geradoras

Lembremos que $H \subseteq G$ é um subgrafo ***gerador*** se $V(H) = V(G)$.

Definição 4.2: árvore geradora

Se uma árvore $T \subseteq G$ é um subgrafo gerador de G, dizemos que T é uma ***árvore geradora*** de G.

Todo grafo conexo tem uma árvore geradora dele: seja G um grafo conexo e seja $T \subseteq G$ um subgrafo gerador de G, conexo e com o menor número de arestas possível. Se T contém circuito então para qualquer aresta e de um circuito $T - e$ é conexo (Prop. 1.3) e tem menos arestas que T, uma contradição. Desse fato, podemos concluir que a recíproca do lema 4.1 do seguinte modo: suponha que G é conexo, com n vértices e $n - 1$ arestas. Seja $T \subseteq G$ uma árvore geradora de G. Como acabamos de mostrar T tem $n - 1$ arestas, logo $T = G$, portanto G é uma árvore.

Teorema 4.2

Um grafo com n vértices é uma árvore se, e somente se, é conexo e o número de arestas é $n - 1$. □

Um algoritmo que recebe um grafo conexo e determina uma árvore geradora do grafo conexo é fácil e rápido e já sabemos como fazer: é o subgrafo induzido pelo conjunto de arestas $M = \{\{v, p[v]\} : v \in V(G) \text{ e } p[v] \neq \text{nil}\}$, onde o vetor $p[\ ]$ é determinado pela execução de uma algoritmo de busca (e.g., Algoritmo 3).

O exercício 60 item 4), baseado no próximo resultado, também nos dá um modo de construir uma árvore geradora de G: começando com $U = \{v\}$, tomamos sucessivamente um extremo fora de U de uma aresta em $\nabla_G(U)$ e acrescentamos o a U.

Proposição 4.2

Um grafo G é conexo se e somente se $\nabla_G(U)) \neq \emptyset$ para todo subconjunto próprio e não-vazio de vértices $U \subsetneq V(G)$.

Demonstração. Seja G um grafo conexo. Se G é conexo e admite um subconjunto próprio e não-vazio $U \subset V(G)$, então existe em G um caminho

$$u = x_0, x_1, \ldots, x_k = w$$

para algum $u \in U$ e algum $w \in \overline{U}$. Tome $m \coloneqq \max\{j : x_j \in U\}$ e temos $0 \leq m < k$ pela escolha de u e de w, logo $\{x_m, x_{m+1}\} \in \nabla_G(U)$.

Para a recíproca, considere um grafo G tal que para todo subconjunto próprio e não-vazio de vértices $U \subsetneq V(G)$ vale $\nabla_G(U)) \neq \emptyset$.

Se G não é conexo, então existem vértices u e w de G sem nenhum caminho entre eles. Defina U como a componente conexa de u. Logo $U \neq \emptyset$, e $w \notin U$, ou seja, $U \subsetneq V(G)$. Pela hipótese, $\nabla_G(U) \neq \emptyset$, e portanto existe uma aresta $\{a, b\} \in E(G)$ com $a \in U$ e $b \in \overline{U}$. Como $a \in U$, existe um caminho P de u a a, portanto, P, b é um caminho de u a b, logo $b \in U$. Assim, $b \in U$ e $b \in \overline{U}$ é uma contradição. Logo G é conexo. □

Exercícios

Exercício 62. Determine o número de arestas de uma floresta com n vértices e com k componentes conexas, em função de n e k.

Exercício 63. Desenhe todas as árvores não-isomorfas com 5 vértices e todas as árvores não-isomorfas com 7 vértices e com grau máximo pelo menos 4.

Exercício 64. Mostre que toda árvore T tem pelo menos $\Delta(T)$ folhas.

Exercício 65. Prove que o conjunto de arestas $\{p[v], v\}$ definido por uma busca em profundidade rotulada induz uma árvore.

Exercício 66. Seja G um grafo. Mostre que as seguintes afirmações são equivalentes:

(a) G é conexo e $|E(G)| = |V(G)| - 1$;

(b) $|E(G)| = |V(G)| - 1$ e G não contém circuito.

(c) G é uma árvore;

Exercício 67. Considere uma floresta F com n vértices e m arestas.

(a) Qual é o número máximo de vértices num componente conexo de F?

(b) Mostre que há pelo menos $\max\{n-2m, 0\}$ vértices de grau zero.

(c) Mostre que se $m < n/2$ então resta pelo menos um vértice de grau zero.

Exercício 68. Seja T uma árvore, u e v dois vértices não adjacentes em T e $a \in E(T)$ uma aresta do circuito em $T + uv$. Mostre que

$$T + uv - a := \big(V(T), E(T) \cup \{\{u,v\}\} \setminus \{a\}\big)$$

é uma árvore. Dizemos que essa árvore foi obtida por uma ***operação elementar*** a partir da árvore T.

Exercício 69. Sejam $T_1, T_2 \subset G$ árvores geradoras distintas de um grafo conexo G. Mostre que T_2 pode ser obtida a partir de T_1 através de uma sequência de operações elementares.

Exercício 70. Em um grafo conexo G dizemos que uma aresta e é uma ***ponte*** se $G - e$ é desconexo. Prove uma ponte pertence a todas as árvores geradoras de G.

Exercício 71. Seja T uma árvore de ordem t. Mostre que qualquer grafo com grau mínimo pelo menos $t-1$ contém um subgrafo isomorfo a T.

Exercício 72 (*precisa de álgebra linear*). Seja G um grafo conexo com n vértices e m arestas, $m \geq n-1$, e seja $T \subset G$ uma árvore geradora de G. Como $|E(T)| = n-1$, há exatamente $m-n+1$ arestas em $E(G) \setminus E(T)$; denote-as por $e_1, \dots, e_{m-n+1}$. Para cada i, o grafo $T + e_i$ contém um *único* circuito C_i, formado por e_i e pelo único caminho em T entre as pontas de e_i. Chamamos $C_1, \dots, C_{m-n+1}$ de ***circuitos fundamentais de G com respeito a T***.

Dados $A, B \subseteq E(G)$, denote por $A \Delta B$ a diferença simétrica. Considere o conjunto $\wp(E(G))$ das partes de $E(G)$ sobre o corpo $\mathbb{F}_2 = \{0,1\}$, com *soma* $A \Delta B$ e *produto por escalar* $0 \cdot A = \varnothing$, $1 \cdot A = A$.

1. Verifique que $\wp(E(G))$ é um espaço vetorial sobre $\mathbb{F}_2$.
2. Prove que para todo circuito $C \subset G$ existem $b_1, \dots, b_{m-n+1} \in \mathbb{F}_2$ tais que $E(C) = b_1 \cdot E(C_1) \Delta \cdots \Delta\, b_{m-n+1} \cdot E(C_{m-n+1})$.
3. Seja $\mathcal{C}(G)$ o subespaço gerado pelos conjuntos de arestas dos circuitos de G. Prove que $E(C_1), \dots, E(C_{m-n+1})$ formam uma base de $\mathcal{C}(G)$ e conclua que $\dim \mathcal{C}(G) = m-n+1$.

Exercício 73 (*precisa de álgebra linear*). Seja G um grafo conexo com n vértices e m arestas, e seja $T \subset G$ uma árvore geradora de G. Sejam $f_1, \dots, f_{n-1}$ as arestas de T. Para cada j, o grafo $T - f_j$ tem exatamente duas componentes; seja S_j o conjunto de vértices de uma delas e defina

$$D_j = \nabla(S_j).$$

Observe que $D_j \cap E(T) = \{f_j\}$, pois f_j é a única aresta de T entre S_j e $V(G) \setminus S_j$. Chamamos $D_1, \dots, D_{n-1}$ de ***cortes fundamentais de G com respeito a T***.

Considere o espaço vetorial $\wp(E(G))$ sobre $\mathbb{F}_2$ com a soma $A \nabla B$ (exercício anterior).

1. Prove que $\nabla(S)\Delta\nabla(S') = \nabla(S\Delta S')$ para quaisquer $S, S' \subseteq V(G)$. Conclua que o conjunto $\mathcal{D}(G)$ dos cortes de G (incluindo $\nabla(\varnothing) = \varnothing$) é um subespaço de $\wp(E(G))$, chamado ***espaço de cortes*** de G.
2. Prove que para todo corte $D = \nabla(S)$ de G existem $b_1, \dots, b_{n-1} \in \mathbb{F}_2$ tais que $D = b_1 \cdot D_1 \nabla \cdots \nabla\, b_{n-1} \cdot D_{n-1}$.
3. Prove que $D_1, \dots, D_{n-1}$ formam uma base de $\mathcal{D}(G)$ e conclua que $\dim \mathcal{D}(G) = n - 1$.
4. Mostre que $\dim \mathcal{C}(G) + \dim \mathcal{D}(G) = m$, onde $\mathcal{C}(G)$ é o espaço de circuitos do exercício anterior. *Dica*: mostre que, para $e_i \notin E(T)$, vale $f_j \in E(C_i)$ se e somente se $e_i \in D_j$, e que todo circuito intersecta todo corte em um número par de arestas.

Exercício 74 (***Shannon switching game*** *(1952)*). O seguinte jogo foi proposto por Claude Shannon para ser jogado numa grade retangular; descrevemos a versão em um grafo qualquer. O jogo é jogado em um grafo finito G com dois vértices distinguidos, s e t, por dois jogadores, o Positivo (*Short*) e o Negativo (*Cut*), que jogam alternadamente.

No início todas as arestas estão *livres*. Na sua vez, o Positivo *fixa* uma aresta livre e, na sua vez, o Negativo *remove* uma aresta livre; uma aresta fixada ou removida permanece assim até o fim do jogo. O Positivo vence se, em algum momento, existe um caminho de s a t formado por arestas fixadas; caso contrário, isto é, se todas as arestas forem marcadas sem que tal caminho exista, vence o Negativo. Em particular, não há empates. Dizemos que o jogo (G, s, t) é

positivo se o Positivo tem estratégia vencedora, não importando quem começa;

negativo se o Negativo tem estratégia vencedora, não importando quem começa;

neutro se vence quem joga primeiro.

Observe que um jogador nunca é prejudicado por uma jogada extra; logo, se um jogador vence jogando em segundo, vence também jogando em primeiro. Disso decorre que todo jogo é positivo, negativo ou neutro. Prove que se G tem um subgrafo H com $s, t \in V(H)$ que admite duas árvores geradoras (de H) disjuntas nas arestas, então o jogo (G, s, t) é positivo. **Dica.** Seja (T_1, T_2) um par de árvores geradoras de H. Esse par é dito *bom* se num dado momento do jogo se nenhuma aresta de $T_1 \cup T_2$ foi removida e toda aresta de $E(T_1) \cap E(T_2)$ já foi fixada. O Positivo deve manter um par bom antes de cada jogada do Negativo. Se o Negativo remove uma aresta $f \in E(T_1)$, o grafo $T_1 - f$ tem 2 componentes e, como T_2 é conexo, a Prop. 4.2 fornece uma aresta $g \in E(T_2)$ ligando as duas componentes; o Positivo fixa g. Cuide dos casos em que f está fora de $T_1 \cup T_2$, em que g já está fixada, e de quem começa o jogo. Ao final, explique por que o Positivo tem um caminho de s a t de arestas fixadas.

Lehman (1964) de forma teórica e genérica, na teoria de matroides, provou que uma estratégia vencedora para o Positivo existe; o jogo de Shannon depende da estrutura de independência linear dos seus circuitos. Bruno e Weinberg (1970) simplificaram e traduziram os conceitos abstratos de matroides em estruturas puras de teoria dos grafos, oferecendo um algoritmo eficiente (de tempo polinomial) para determinar quem vence. A metodologia impactou o projeto de circuitos elétricos e redes de comutação além de servir de base para algoritmos de otimização de fluxos em redes e conectividade em grafos.

4.1 Árvores geradoras de custo mínimo em grafos com pesos nas arestas

Seja $G = (V, E, \rho)$ um grafo conexo com pesos $\rho : E \to \mathbb{R}$ nas arestas. Para $F \subseteq E$ definimos o ***custo*** de F por

$$c(F) = \sum_{e \in F} \rho(e),$$

e o custo de um subgrafo $H \subseteq G$ é $c(H) := c(E(H))$. O problema que nos interessa a partir de agora é: dado $G = (V, E, \rho)$, determinar uma árvore geradora T de G tal que

$$c(T) = \min\{c(T') : T' \text{ é árvore geradora de } G\}.$$

Uma tal árvore é chamada de ***árvore geradora mínima*** de G. Ela existe, pois G é conexo (logo tem alguma árvore geradora) e tem apenas um número finito delas; porém pode não ser única.

Observação 4.1: um cuidado com pesos negativos

Se $\rho \geq 0$, o subgrafo gerador conexo de menor custo pode ser tomado como uma árvore: se ele contém um circuito, removemos uma aresta do circuito, o que mantém a conexidade e não aumenta o custo. Com pesos negativos isso falha: num triângulo com pesos -1, o triângulo inteiro custa -3, mais barato que cada árvore geradora cujo custos é -2.

A seguir apresentamos os algoritmos de Dijkstra–Jarník–Prim[1] e de Kruskal para determinar uma árvore geradora mínima. Ambos são algoritmos gulosos: a cada passo escolhem, dentre as opções disponíveis, a que parece melhor naquele momento (ótimo local), sem reconsiderar escolhas anteriores. Em geral essa estratégia não produz uma solução ótima (ótimo global), mas para árvores geradoras mínimas ela produz, e é isso que provaremos.

1: O algoritmo de Dijkstra–Jarník–Prim foi descoberto mais de uma vez, em contextos distintos. Foi descrito primeiro por Vojtěch Jarník (1930), que considerou complicada a solução de Otakar Borůvka (1926), motivada pela eletrificação rural da Morávia. Robert Prim o redescobriu em 1957 nos Bell Labs, ao tratar de redes de telefonia, e Edsger Dijkstra em 1959, a pedido de engenheiros do Mathematisch Centrum, em Amsterdã, interessados em reduzir a fiação de um computador.

4.1.1 Algoritmo de Dijkstra–Jarník–Prim para árvore geradora mínima

O algoritmo de Dijkstra–Jarník–Prim recebe um grafo conexo $G = (V, E, \rho)$ e devolve $S \subseteq E$ tal que $G[S]$ é uma árvore geradora mínima de G. O algoritmo parte de um conjunto unitário, digamos $U = \{v\}$ para algum $v \in V$, e a cada rodada o algoritmo acrescenta a U o vértice de $\overline{U}$ que é extremo de uma aresta de menor custo no corte $\nabla_G(U)$ (que existe pela Proposição 4.2 e empates são desfeitos arbitrariamente), até que $U = V$. No final, as arestas escolhidas induzem uma árvore geradora mínima.

Algoritmo 7: Dijkstra–Jarník–Prim(G)

Dado : um grafo conexo G com pesos nas arestas.
Devolve : uma árvore geradora mínima.

1 $U, S = \emptyset$;
2 escolha $v \in V(G)$ arbitrário;
3 $U \leftarrow \{v\}$;
4 **enquanto** $U \neq V(G)$ **faça**
5 escolha $\{u, w\} \in \nabla_G(U)$ de peso mínimo;
6 insira $\{u, w\}$ em S;
7 $v \leftarrow \{u, w\} \cap \overline{U}$;
8 insira v em U;
9 **retorne** (V, S).

Notemos que esse algoritmo é uma ligeira modificação do Algoritmo 5 (pela Proposição 4.2 os laços desses dois algoritmos têm a mesma condição).

Análise e correção do algoritmo de Dijkstra–Jarník–Prim.

Usaremos a técnica do invariante do laço.

Proposição 4.3

Seja $n = |V(G)|$ e denote por U_k e S_k os valores de U e S após a k-ésima iteração do `enquanto` da linha 4 (com $U_0 = \{v\}$ e $S_0 = \emptyset$). Para $0 \leq k \leq n-1$, o par $T_k := (U_k, S_k)$ é uma árvore e é subgrafo de alguma árvore geradora mínima de G.

Demonstração. Por indução em k. Para $k = 0$, T_0 é a árvore trivial com o vértice v, subgrafo de toda árvore geradora de G; em particular de uma mínima, que existe pois G é conexo e tem finitas árvores geradoras.

Seja $1 \leq k \leq n-1$ e suponha que T_{k-1} é uma árvore contida em uma árvore geradora mínima T. Na k-ésima iteração o algoritmo escolheu (linha 5) uma aresta $\{u, w\} \in \nabla_G(U_{k-1})$ de peso mínimo no corte, com $u \in U_{k-1}$ e $w \notin U_{k-1}$, e fez $U_k = U_{k-1} \cup \{w\}$ e $S_k = S_{k-1} \cup \{\{u, w\}\}$. Como w é um vértice novo ligado a T_{k-1} apenas por $\{u, w\}$, T_k é conexo e nenhum circuito passa por w (que tem grau 1); logo T_k é uma árvore.

Se $\{u, w\} \in E(T)$, então $T_k \subseteq T$ e nada há a provar. Suponha $\{u, w\} \notin E(T)$. Pelo Teorema 4.1, item (2), existe um único caminho P em T de u a w. Como $u \in U_{k-1}$ e $w \notin U_{k-1}$, o argumento da primeira metade da prova da Proposição 4.2 mostra que P contém uma aresta $e \in \nabla_T(U_{k-1})$. Temos $e \neq \{u, w\}$ (pois $\{u, w\} \notin E(T)$) e

(a) $T' := T - e + uw$ é uma árvore geradora de G: como $e \in E(P)$, os vértices u e w estão em componentes distintas de $T - e$, que $\{u, w\}$ religa; T' é conexo e tem $n-1$ arestas.

(b) $e \notin S_{k-1}$: toda aresta de S_{k-1} tem as duas pontas em U_{k-1}, e e tem uma ponta fora de U_{k-1}. Assim $S_{k-1} \subseteq E(T) \setminus \{e\} \subseteq E(T')$ e, como $\{u, w\} \in E(T')$, $T_k \subseteq T'$.

(c) Como $e \in \nabla_G(U_{k-1})$ e $\{u, w\}$ tem peso mínimo nesse corte, $\rho(\{u, w\}) \leq \rho(e)$, logo $c(T') = c(T) - \rho(e) + \rho(\{u, w\}) \leq c(T)$. Pela minimalidade de T, T' também é uma árvore geradora mínima.

Assim T_k é subgrafo da árvore geradora mínima T'. A proposição segue do Princípio da Indução Finita. □

Corolário 4.1

O algoritmo de Dijkstra–Jarník–Prim devolve o conjunto S de arestas de uma árvore geradora mínima de G.

Demonstração. Cada iteração acrescenta um vértice a U, logo o laço termina após $n - 1$ iterações com $U = V$. Pela proposição, (V, S) é uma árvore contida em uma árvore geradora mínima T. Ambas têm $n - 1$ arestas, portanto $(V, S) = T$. □

A complexidade de tempo do algoritmo de Dijkstra–Jarník–Prim depende da implementação do passo 5. Mantendo um vetor booleano que diz quais vértices estão em U, podemos testar em $O(1)$ se uma aresta pertence a $\nabla_G(U)$ e, a cada iteração, percorrer todas as arestas para encontrar uma de menor peso nesse corte, o que custa $O(|E|)$. Como há $|V| - 1$ iterações, o tempo total é $O(|V||E|)$. (Ordenar as arestas antes não melhora essa cota, pois a busca da primeira aresta do corte ainda pode percorrer todas elas.) Com mais cuidado, podemos melhorar o tempo para grafos esparsos: uma implementação com *heap* binária tem complexidade $O(|E| \log |V|)$.

Heap binária e análise de complexidade do algoritmo de Prim

Uma ***heap*** binária para mínimo é uma estrutura de dados que mantém um conjunto de elementos, cada um com uma *chave*, e permite acessar rapidamente o elemento de menor chave. Ela implementa uma ***fila de prioridade*** quando a menor chave é a de maior prioridade.

Formalmente, é uma árvore binária *quase completa*, isto é, com todos os níveis cheios, exceto possivelmente o último, que é preenchido da esquerda para a direita, e que satisfaz a ***propriedade de heap***: *a chave no nó pai é menor ou igual à chave nos seus filhos.* Em consequência, a raiz contém sempre o elemento de menor chave.

Por ser quase completa, a árvore é armazenada em um vetor A, sem ponteiros. Para o nó na posição i:

- pai: $\lfloor i/2 \rfloor$;
- filho esquerdo: $2i$;
- filho direito: $2i + 1$.

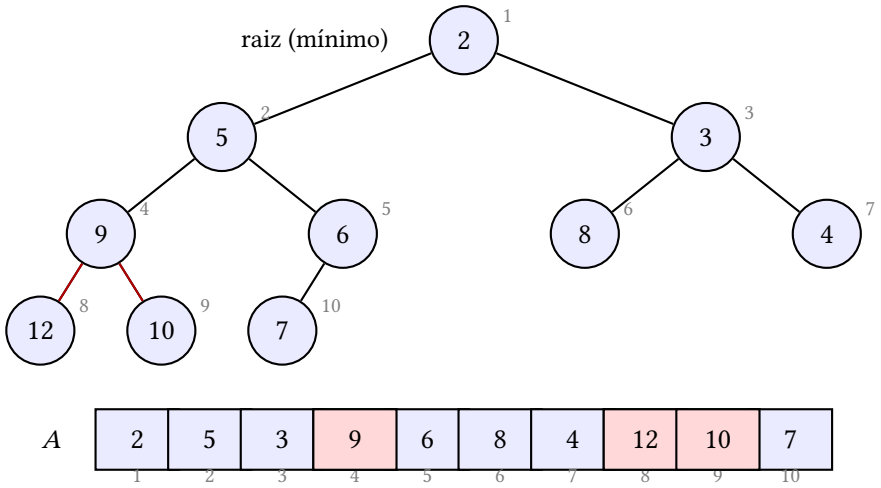


Figura 4.2: *Heap* com 10 elementos e sua representação em vetor. Para o nó na posição i, o pai está em $\lfloor i/2 \rfloor$ e os filhos em $2i$ e $2i+1$. Em destaque, o nó $i = 4$ e seus filhos 8 e 9.

A altura da árvore é $\lfloor \log_2 n \rfloor$, que dá custo logarítmico das operações (Tabela 4.1).

Tabela 4.1: Operações do *Heap* Mínimo e seus custos. Depois de uma operação pode ser necessário trocar chaves de posição para refazer a propriedade de *heap*. Isso significa *subir* uma chave trocando de posição como o pai ou *descer* uma chave trocando de posição como o menor filho. Para que `diminuir-chave` seja eficiente, é preciso saber em que posição do vetor está cada elemento. Isso se resolve mantendo um vetor auxiliar pos$[v]$, atualizado a cada troca.

Operação	Descrição	Custo
`min`	devolve $A[1]$	$O(1)$
`inserir`(x)	coloca x na última posição e refaz *heap*	$O(\log n)$
`extrair-min`	remove $A[1]$, move o último elemento para a raiz e refaz *heap*	$O(\log n)$
`diminuir-chave`(i,k)	reduz a chave da posição i para k e refaz *heap*	$O(\log n)$
`construir-heap`	organiza um vetor arbitrário em forma de *heap*	$O(n)$

As operações preservam a propriedade de *heap* da seguinte forma.

- *Subir uma chave na árvore* — ocorre nas operações `inserir` e `diminuir-chave`: antes da operação a propriedade de *heap* vale; depois, a única violação possível é entre o nó x modificado e seu pai. Se tal violação ocorre, trocamos os dois. Após a troca, x é menor que o antigo pai, que era menor ou igual ao irmão de x e aos filhos de x, de modo que a única violação possível é, de novo, entre x e seu novo pai, um nível acima. O processo termina na raiz, ou quando não há mais violação, em no máximo $\lfloor \log_2 n \rfloor$ passos.

- *Descer uma chave na árvore* — ocorre nas operações `extrair-min` e `construir-heap`): aqui supomos que as subárvores dos filhos de x já são *heap*; então a única violação possível é entre um nó x e seus filhos. Se x é maior que algum filho, trocamos x com o *menor* dos filhos c; assim, o novo pai c é menor que x e menor ou igual ao outro filho. A única violação

possível passa a ser entre x e seus novos filhos, um nível abaixo. O processo termina quando não há violação ou ao chegar em uma folha. Depois dele, a subárvore de x é uma *heap*.

Dado um vetor com n elementos em ordem qualquer, construímos nesse vetor uma *heap* organizando-o de baixo para cima. Os nós internos ocupam as posições $1, \ldots, \lfloor n/2 \rfloor$ e as folhas ocupam as demais, que já são *heaps*. Processamos cada $i = \lfloor n/2 \rfloor, \ldots, 1$, nessa ordem, aplicando "*descer*" ao nó i; ao processá-lo, as subárvores de $2i$ e $2i+1$ já são *heap*, e depois a subárvore de i também é. No final, o vetor satisfaz a propriedade de *heap*.

A ***altura de um nó*** é o maior número de arestas de um caminho descendente dele até uma folha. Descer um nó de altura h custa $O(h)$ e há no máximo $\lceil n/2^{h+1} \rceil \le n/2^h$ nós de altura h (veja Cormen et al. (2012); a desigualdade vale para $h \le \lfloor \log_2 n \rfloor$, pois $n/2^{h+1} \ge 1/2$). Como $\sum_{h \ge 0} h/2^h = 2$, o custo total para organizar um vetor qualquer numa *heap* é

$$\le \sum_{h=0}^{\lfloor \log_2 n \rfloor} \frac{n}{2^h} O(h) = O\left(n \sum_{h \ge 0} \frac{h}{2^h} \right) = O(n).$$

Algoritmo 8: Dijkstra–Jarník–Prim—heap(G)

Dado : um grafo conexo $G = (V, E, \rho)$ com pesos nas arestas.

Devolve : uma árvore geradora mínima.

```
1  S ← ∅;
2  escolha v ∈ V arbitrário;
3  para cada z ∈ V faça
4      chave[z] ← ∞;
5      pai[z] ← nil;
6  chave[v] ← 0;
7  A ← construir-heap(V)
8  enquanto A ≠ ∅ faça
9      w ← extrair-min(A)
10     se pai[w] ≠ nil então insira {pai[w], w} em S;
11     para cada z ∈ N_G(w) faça
12         se z ∈ A e ρ({w, z}) < chave[z] então
13             pai[z] ← w;
14             diminuir-chave(A, z, ρ({w, z}));
15 retorne (V, S)
```

O ponto crítico do algoritmo de Dijkstra–Jarník–Prim é encontrar, a cada iteração, a aresta mais barata que sai de U. Para isso, guardamos os vértices fora de U em uma *heap* A ordenada por `chave`, isto é, $U = V \setminus A$, de modo que, para $z \in A$,

$$\text{chave}[z] = \min\{\rho(\{u, z\}) : u \in U,\ \{u, z\} \in E(G)\}$$

(ou ∞ se z não tem vizinho em U) e pai$[z]$ é um vértice $u \in U$ que realiza esse mínimo. Esse invariante vale após cada iteração do laço da linha 8 do Algoritmo 8, a partir da primeira (que extrai o vértice inicial v, de chave 0 por convenção, e faz $U = \{v\}$). De fato, quando w entra em U só mudam as chaves dos vizinhos $z \in A$ de w, e o teste $\rho(\{w, z\}) <$ chave$[z]$ as atualiza para o mínimo. Consequentemente, a partir da segunda iteração, seja U o conjunto antes da extração; como G é conexo e $\varnothing \neq U \subsetneq V$, a Proposição 4.2 dá $\nabla_G(U) \neq \varnothing$, e o vértice w extraído tem chave$[w] < \infty$: w é o extremo em $\overline{U}$ de uma aresta $\{\text{pai}[w], w\}$ de menor peso em $\nabla_G(U)$, que é exatamente o passo da linha 5.

Supomos o *grafo dado por listas de adjacência* e mantemos pos$[z]$ (com `nil` para os vértices já extraídos), o que permite testar $z \in A$ em $O(1)$. A inicialização e a construção da *heap* custam $O(|V|)$. As $|V|$ chamadas a `extrair-min` custam $O(|V| \log |V|)$. Cada aresta $\{w, z\}$ é examinada duas vezes (a partir de cada extremo), mas só provoca uma chamada a `diminuir-chave` quando o outro extremo ainda está em A, ou seja, no máximo uma vez, o que dá $O(|E|)$ chamadas de custo $O(\log |V|)$ cada. Como G é conexo, $|V| \leq 2|E|$ (para $|V| \geq 2$), e todos os termos ficam absorvidos por

$$O(|E| \log |V|).$$

Para comparação, se a *heap* for substituída por um vetor simples, `extrair-min` custa $O(|V|)$ e `diminuir-chave` custa $O(1)$, resultando em $O(|V|^2)$. Essa versão é melhor em grafos densos ($|E| = \Theta(|V|^2)$), enquanto a *heap* binária é melhor em grafos esparsos.

Teorema 4.3

O algoritmo Dijkstra–Jarník–Prim—heap(G) computa uma árvore geradora mínima de um grafo conexo (V, E) com pesos nas arestas em tempo $O(|E| \log |V|)$.

4.1.2 Algoritmo de Kruskal para árvore geradora mínima

O algoritmo de Kruskal, proposto por Joseph Kruskal em 1956, também é bastante simples: a cada passo escolhe-se a aresta mais barata dentre as que ainda não foram examinadas, desde que ela não forme circuito com as arestas já escolhidas. Arestas de mesmo peso podem ser ordenadas arbitrariamente.

Algoritmo 9: Kruskal(G)

Dado : um grafo conexo G com pesos nas arestas.
Devolve : árvore geradora de custo mínimo.

```
1 S ← ∅;
2 F ← fila das arestas em ordem crescente de peso;
3 para cada e ∈ F faça
4     se (V, S ∪ {e}) é acíclico então
5         insira e em S;
6 retorne (V, S).
```

Correção do algoritmo de Kruskal.

Vamos provar que o algoritmo de Kruskal determina uma árvore geradora de custo mínimo de um grafo conexo e com pesos nas arestas.

Seja S o conjunto devolvido e $(e_1, \dots, e_k)$ as arestas de S na ordem em que foram inseridas. Como S começa vazio e uma aresta só é inserida se S continua acíclico (linha 4), o grafo (V, S) é acíclico. Seja T uma árvore geradora mínima de G com o maior número possível de arestas em comum com S. Mostraremos que $E(T) = S$.

Vamos provar que $S \subseteq T$. Suponha que não e seja $m := \min\{j : e_j \in S \setminus E(T)\}$. Então $e_1, \dots, e_{m-1} \in E(T)$. Pelo Teorema 4.1, $T + e_m$ contém um circuito C, que contém e_m. Como (V, S) é acíclico, C tem uma aresta $f \notin S$; em particular $f \neq e_m$, logo $f \in E(T)$. Considere

$$R := \{e_1, \dots, e_{m-1}\} \cup \{f\} \subseteq E(T),$$

que, por estar contido em uma árvore, é acíclico.

Afirmamos que f vem depois de e_m na fila F. Se viesse antes, no momento em que f foi examinada o conjunto S corrente estaria contido em $\{e_1, \dots, e_{m-1}\}$, e f só teria sido rejeitada se esse conjunto mais f contivesse circuito; mas este está contido em R, que é acíclico. Como F está em ordem crescente de peso, $\rho(f) \geq \rho(e_m)$.

Como f está no único caminho de T entre as pontas de e_m, essas pontas estão em componentes distintas de $T - f$, e portanto $T' := T - f + e_m$ é uma árvore geradora de G, com

$$c(T') = c(T) - \rho(f) + \rho(e_m) \leq c(T).$$

Pela minimalidade de T, T' também é mínima. Mas $e_m \in S$ e $f \notin S$, logo T' tem uma aresta a mais em comum com S do que T, contrariando a escolha de T. Portanto $S \subseteq E(T)$.

Vamos provar que $E(T) \subseteq S$. Suponha $e \in E(T) \setminus S$. No momento em que e foi examinada, o conjunto S' corrente satisfazia $S' \subseteq S \subseteq E(T)$, logo $S' \cup \{e\} \subseteq E(T)$ é acíclico e o algoritmo teria inserido e em S, uma contradição.

Assim $S = E(T)$, isto é, $(V, S) = T$ é uma árvore geradora mínima.

Teorema 4.4: correção do algoritmo de Kruskal

Para todo grafo conexo $G = (V, E, \rho)$, o algoritmo de Kruskal devolve (V, S) tal que (V, S) é uma árvore geradora mínima de G. □

Análise da complexidade do algoritmo de Kruskal.

Notemos que enquanto o conjunto S evolui de $\emptyset$ até definir uma árvore geradora ele sempre define uma subfloresta geradora de G, e seus componentes formam uma partição de V. Para $e = \{x, y\} \notin S$ vale

$(V, S \cup \{e\})$ é acíclico $\iff$ x e y estão em componentes distintos de (V, S).

De fato, se estão no mesmo componente, um caminho de x a y em (V, S) junto com e é um circuito. Se estão em componentes distintos e houvesse um circuito em $(V, S \cup \{e\})$, ele conteria e (pois (V, S) é acíclico) e o restante dele seria um caminho de x a y em (V, S), absurdo.

O teste na linha 4 acima verifica se a aresta $e = \{x, y\}$ não liga vértices do mesmo conjunto (caso contrário, teremos um circuito) e, nesse caso, na linha 5, une o conjunto onde está o vértice x com o conjunto do vértice y.

Assim, precisamos de estruturas de dados que, dinamicamente, representem e manipulem conjuntos disjuntos de modo eficiente. Estruturas como essa são conhecidas na literatura como estruturas para união-e-busca (*union-find*) ou *estruturas de dados para conjuntos disjuntos*.

Essas estruturas mantêm uma família de subconjuntos disjuntos com um elemento de cada subconjunto eleito como o *representante* do subconjunto e permitem as operações

`cria`(x) cria o conjunto unitário $\{x\}$, com representante x;
`busca`(x) devolve o representante do conjunto ao qual x pertence;
`uniao`(x, y) substitui os conjuntos que contém x e y pela união desses conjuntos (e determina um representante para essa união).

Reescrevendo o algoritmo de Kruskal com as considerações feitas acima sobre estruturas de dados para conjuntos disjuntos:

Algoritmo 10: Kruskal(G)

Dado : um grafo conexo G com pesos nas arestas.
Devolve : árvore geradora de custo mínimo.

```
1 S ← ∅;
2 F ← fila das arestas em ordem crescente de peso;
3 para cada v ∈ V(G) faça cria(v) ;
4 para cada {u, v} ∈ F faça
5     se busca(u) ≠ busca(v) então
6         insira {u, v} em S;
7         uniao(u, v)
8 retorne (V, S)
```

Entre as estruturas de dados mais eficientes para implementar conjuntos disjuntos destaca-se a representação por floresta com as heurísticas ***união por rank*** e ***busca com compressão de caminhos***, para a qual m operações sobre n elementos custam $O(m\,\alpha(n))$, onde α é a inversa da função de Ackermann, que cresce tão lentamente que é ≤ 4 para qualquer n prático Cormen et al. (2012).

No Kruskal há $|V|$ operações `cria`, no máximo $2|E|$ operações `busca` e no máximo $|V|-1$ operações `uniao`, no total menos que $2(|E|+|V|) = O(|E|)$. Assim o custo das operações é dominado pela ordenação das arestas, e o tempo total é $O(|E|\log|E|) = O(|E|\log|V|)$, pois $|E| \leq |V|^2$. A seguir analisamos uma estrutura bem simples.

Usamos um vetor C indexado por $V = \{1, \ldots, n\}$ e a ideia é que dois elementos $x, y \in V$ estão no mesmo conjunto se e somente se tal que dois elementos x, y estão no mesmo conjunto se e somente se $C[x] = C[y]$. Escolhemos como representante o *menor elemento* do conjunto, isto é, $C[x]$ é o menor elemento do conjunto de x (esse é o invariante mantido pelas operações abaixo).

Algoritmo 11: cria(x)

```
1 C[x] ← x;
```

Algoritmo 12: busca(x)

```
1 retorne C[x]
```

Algoritmo 13: uniao(x, y)

```
1 a ← busca(x);
2 b ← busca(y);
3 k ← min{a, b};
4 para cada i ∈ {1, ..., n} faça
5     se C[i] = a ou C[i] = b então C[i] ← k ;
```

O invariante é preservado: k é o menor elemento da união, pois a e b são os menores elementos dos dois conjuntos. (É importante guardar a e b antes do laço: os valores $C[x]$ e $C[y]$ mudam durante ele.)

Lema 4.1

O tempo gasto com m operações num universo V com n elementos é $O(mn)$.

Demonstração. As operações `cria` e `busca` custam $O(1)$ e `uniao` custa $\Theta(n)$, pois o laço percorre sempre as n posições de C. □

Em particular o limite é justo: toda `uniao` custa $\Theta(n)$, independentemente da sequência.

Corolário 4.2

O tempo do algoritmo Kruskal (G) com vetor para representar conjuntos disjuntos é $O(|E| \log |E| + |V|^2)$, em particular $O(|V| \cdot |E|)$, para todo grafo conexo G com pesos nas arestas.

Demonstração. A ordenação custa $O(|E| \log |E|)$. Há $|V|$ operações `cria` e no máximo $2|E|$ operações `busca`, de custo $O(1)$ cada, e no máximo $|V| - 1$ operações `uniao` (cada uma reduz em 1 o número de componentes), de custo $O(|V|)$ cada. O total é $O(|E| \log |E| + |E| + |V|^2)$. Como G é conexo, $|E| \geq |V| - 1$, logo $|V|^2 = O(|V||E|)$; e como $\log |E| \leq 2 \log |V| \leq 2|V|$, $|E| \log |E| = O(|V||E|)$. □

4.1.3 Estruturas de dados para representar conjuntos disjuntos

Nessa seção veremos estruturas mais eficientes para implementar as três operações utilizadas no algoritmo de Kruskal. Para compará-las, analisamos o tempo gasto por uma sequência de m operações, e não o pior caso de cada operação isolada, pois este costuma superestimar o custo da maioria delas (análise amortizada). Supomos que as n primeiras operações são `cria`, uma para cada elemento do universo $\mathcal{U} = \{1, \ldots, n\}$; em particular, $m \geq n$.

O problema é representar dinamicamente uma família $\mathcal{S}$ de conjuntos disjuntos $\mathcal{C} = \{C_1, C_2, \ldots, C_k\}$ sobre um universo $\mathcal{U}$ tal que cada conjunto é identificado por um *representante*, que é um elemento do próprio conjunto. Dado que o conjunto não mudou, toda vez que se pergunta pelo representante a resposta deve ser a mesma. Para as nossas aplicações podemos assumir que os elementos são de um universo $\mathcal{U} = \{1, 2, \ldots, n\}$.

Essas estruturas devem comportar as operações `cria`, `busca` e `uniao` sobre $\mathcal{C}$:

`cria`(x) para x que ainda não pertence a nenhum conjunto, acrescenta $\{x\}$ à família $\mathcal{C}$;

`busca`(x) devolve o representante do conjunto ao qual x pertence;

`uniao`(x, y) para x e y em conjuntos distintos C_x e C_y, remove C_x e C_y de $\mathcal{C}$, acrescenta $C_x \cup C_y$ e escolhe um representante para ele.

Representação usando listas ligadas

Cada conjunto é dado por uma lista ligada cujo primeiro nó é o seu representante, e um vetor $V[1..n]$ guarda, para cada x, um ponteiro para o nó de x (`nil` se x ainda não foi criado). Cada nó tem os atributos `prox` e `rep` (ponteiro para o representante); o nó representante guarda ainda o número de elementos ℓ da lista e um ponteiro `ult` para o seu último nó.

`cria`(x) cria um nó z com `rep`$[z] = z$, $\ell = 1$, `ult` $= z$, e faz $P[x] \leftarrow z$;

`busca`(x) devolve $P[x]$.`rep`, em tempo $O(1)$;

`uniao`(x, y) sejam L_x, L_y as listas de x e y, com L_x a de maior tamanho (se $\ell[L_x] < \ell[L_y]$ troque os papéis). Percorre-se L_y atualizando `rep` de cada nó para o representante de L_x, liga-se `ult`$[L_x]$ ao primeiro nó de L_y e atualizam-se ℓ e `ult` do representante de L_x, que é o da união. O custo é $O(1 + \min\{\ell[L_x], \ell[L_y]\})$.

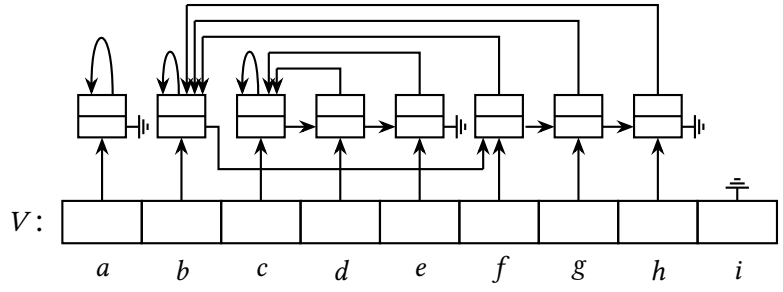


Figura 4.3: Representação com listas ligadas dos subconjuntos $\{c, d, e\}$, $\{a\}$ e $\{b, f, g, h\}$ do universo $\{a, b, c, d, e, f, g, h, i\}$.

Uma busca é feita em tempo constante e o custo de uma união é basicamente o custo das atualizações. Sem outra precaução, $n - 1$ uniões podem custar $\Theta(n^2)$ (basta sempre atualizar a lista maior), o mesmo que a representação por vetor. Adotando a seguinte heurística, a complexidade melhora substancialmente:

atualizar sempre a menor lista: numa união, os ponteiros `rep` são atualizados na menor das duas listas (em qualquer uma delas, se têm o mesmo tamanho), que é acrescentada ao final da maior. (‡)

Para um elemento x do universo, a primeira vez que `rep`$[x]$ é atualizado resulta numa lista de tamanho 2. A segunda vez que `rep`$[x]$ é atualizado resulta numa lista de tamanho 4 e assim por diante. Logo, o número de atualizações que `rep[ ]` de um elemento qualquer é atualizado é $\log_2 n$.

Proposição 4.4

Supondo (‡), em qualquer momento, se `rep`$[x]$ foi atualizado k vezes, então a lista que contém x tem pelo menos 2^k elementos.

Demonstração. Os conjuntos só crescem, logo o tamanho da lista de x não diminui com o tempo. Mostramos por indução em k que, logo após a k-ésima atualização de `rep`$[x]$, a lista de x tem pelo menos 2^k elementos; por monotonicidade isso vale também depois. Para $k = 0$, basta que a lista de x tenha pelo menos $1 = 2^0$ elemento. Seja $k \geq 1$ e seja L a lista de x imediatamente antes da k-ésima atualização; pela hipótese de indução e pela monotonicidade, $|L| \geq 2^{k-1}$. Como `rep`$[x]$ é atualizado, por (‡) a outra lista L' tem $|L'| \geq |L|$, e a lista resultante tem $|L| + |L'| \geq 2|L| \geq 2^k$ elementos. □

Corolário 4.3

Supondo (‡), o representante de um elemento qualquer muda no máximo $\lfloor \log_2 n \rfloor$ vezes.

Demonstração. Se muda k vezes, a lista de x tem pelo menos 2^k elementos e no máximo n, logo $2^k \leq n$. □

Teorema 4.5

Supondo (‡) e que as n primeiras operações são `cria`, o tempo para m operações num universo com n elementos é $O(m + n \log n)$.

Demonstração. Cada `cria` e cada `busca` custa $O(1)$, e cada `uniao` custa $O(1)$ mais o número de ponteiros `rep` atualizados. Pelo corolário, cada um dos n elementos tem `rep` atualizado no máximo $\lfloor \log_2 n \rfloor$ vezes, e portanto o total de atualizações é $O(n \log n)$. Somando, o tempo é $O(m + n \log n)$. □

Corolário 4.4

Para todo grafo simples conexo G com pesos nas arestas, o tempo do algoritmo de Kruskal com listas ligadas e (‡) é $O(|E(G)| \log |V(G)|)$.

Demonstração. A ordenação das arestas custa $O(|E| \log |E|)$, mas $|E| \leq |V|^2$, logo ordenar custa $O(|E| \log |V|)$. O algoritmo faz $|V|$ operações `cria`, no máximo $2|E|$ operações `busca` e no máximo $|V| - 1$ operações `uniao`, ou seja, $m = O(|V| + |E|) = O(|E|)$ operações, pois $|E| \geq |V| - 1$. Pelo teorema com $n = |V|$, elas custam $O(|E| + |V| \log |V|)$, e como $|V| \leq |E| + 1$ o total é $O(|E| \log |V|)$. □

Representação por floresta

A ideia é que cada conjunto seja representado por uma árvore, em que cada elemento é um nó. Cada nó aponta para o seu pai, a raiz é o representante e o pai da raiz é ela mesma (Figura 4.4). Assim, `cria` cria em tempo constante uma árvore com um só nó; `busca`(x) percorre o caminho de x até a raiz, em tempo proporcional à profundidade de x, e devolve a raiz; e a união de duas árvores é feita em tempo constante *a partir das duas raízes*, pendurando uma na outra (Figura 4.5). Portanto, `uniao`(x, y) custa duas buscas mais $O(1)$.

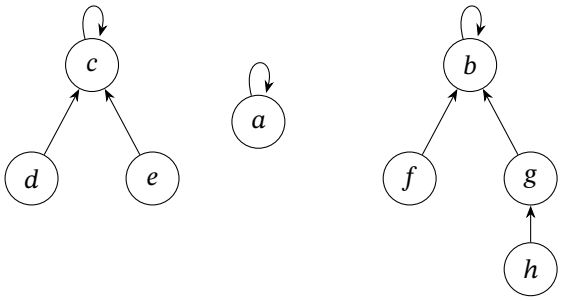


Figura 4.4: Uma representação com árvores dos subconjuntos $\{c, d, e\}$, $\{a\}$ e $\{b, f, g, h\}$.

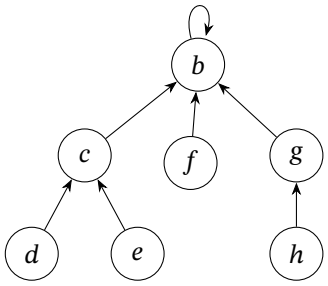


Figura 4.5: Resultado de uniao(e, f).

Essa estrutura pode ser muito ruim: se pendurarmos sempre a raiz da árvore maior numa árvore unitária, formamos uma cadeia com n nós, em que uma busca custa $\Theta(n)$. Com uma heurística simples o resultado melhora substancialmente. O *tamanho* de uma árvore é o seu número de nós.

União por tamanho: *a raiz da árvore de menor tamanho é pendurada na raiz da árvore de maior tamanho (em caso de empate, em qualquer uma).*

Definimos a *altura de uma árvore* como a maior distância de um nó até a raiz e temos a seguinte relação entre tamanho e altura resultantes de uma seqüência de uniões com a heurística de união por tamanho.

Proposição 4.5

Usando união por tamanho, em qualquer momento cada árvore tem pelo menos 2^h nós, onde h é a sua altura.

Demonstração. Por indução no número k de uniões. Para $k = 0$ toda árvore tem um nó e altura 0, e $1 \geq 2^0$. Seja $k \geq 1$ e considere a k-ésima união, de duas árvores distintas A e B, com tamanhos t_A, t_B e alturas

h_A, h_B; pela hipótese de indução $t_A \geq 2^{h_A}$ e $t_B \geq 2^{h_B}$. As demais árvores não mudam. Sem perda de generalidade $t_A \geq t_B$, e então a raiz de B é pendurada na raiz de A. A árvore resultante tem $t' = t_A + t_B$ nós e altura $h' = \max\{h_A,\ h_B+1\}$. Se $h' = h_A$, então $t' \geq t_A \geq 2^{h_A} = 2^{h'}$. Se $h' = h_B + 1$, então $t' = t_A + t_B \geq 2t_B \geq 2 \cdot 2^{h_B} = 2^{h'}$. Em ambos os casos o resultado vale após a k-ésima união. □

Corolário 4.5

Usando união por tamanho, e supondo que as n primeiras operações são `cria`, o tempo de m operações num universo com n elementos é $O(m \log n + n)$.

Demonstração. Pela proposição, uma árvore de altura h tem $2^h \leq n$ nós, logo $h \leq \log_2 n$ e cada `busca` custa $O(1 + \log n)$. Cada `cria` custa $O(1)$ e cada `uniao` custa duas buscas mais $O(1)$, isto é, $O(1 + \log n)$. O total é $O(n + m(1 + \log n)) = O(m \log n + n)$. □

Cada nó tem um atributo pai$[x]$ e, nas raízes, um atributo $t[x]$ que armazena o número de elementos na árvore enraizada em x. Os algoritmos ficam da seguinte forma

Algoritmo 14: cria(x)

```
1 pai[x] ← x;
2 t[x] ← 1.
```

Algoritmo 15: busca(x)

```
1 enquanto x ≠ pai[x] faça
2 |  x ← pai[x];
3 retorne x
```

O procedimento `link` a seguir pressupõe que x e y são raízes distintas, e `uniao` que x e y estão em árvores distintas.

Algoritmo 16: link(x, y)

```
1 t[x] ← t[x] + t[y];
2 pai[y] ← x.
```

Algoritmo 17: uniao(x, y)

```
1 r_x ← busca(x);
2 r_y ← busca(y);
3 se t[r_x] ≥ t[r_y] então link(r_x, r_y);
4 senão link(r_y, r_x);
```

Como o tempo de `busca` é proporcional à altura da árvore, parece mais natural uma heurística baseada na altura e não no tamanho.

União por altura: *a raiz da árvore mais baixa é pendurada na raiz da árvore mais alta (em caso de empate, em qualquer uma).*

Cada raiz r guarda $h[r]$, a altura da árvore enraizada em r, que é iniciada com 0.

Algoritmo 18: cria(x)

```
1 pai[x] ← x;
2 h[x] ← 0;
```

Algoritmo 19: link(x, y)

```
1 se h[x] = h[y] então
2     h[x] ← h[x] + 1
3 pai[y] ← x;
```

Algoritmo 20: uniao(x, y)

```
1 r_x ← busca(x);
2 r_y ← busca(y);
3 se h[r_x] ≥ h[r_y] então link(r_x, r_y);
4 senão link(r_y, r_x);
```

O procedimento `link`(x, y) pressupõe que x e y são raízes distintas com $h[x] \geq h[y]$; `busca` é o mesmo. Só é preciso incrementar $h[x]$ quando as árvores têm a mesma altura. O valor $h[r]$ é a altura exata da árvore de raiz r: se $h_A \geq h_B$ e a raiz de B é pendurada na de A, a nova altura é $\max\{h_A, h_B + 1\}$, que é h_A se $h_A > h_B$ e $h_A + 1$ se $h_A = h_B$.

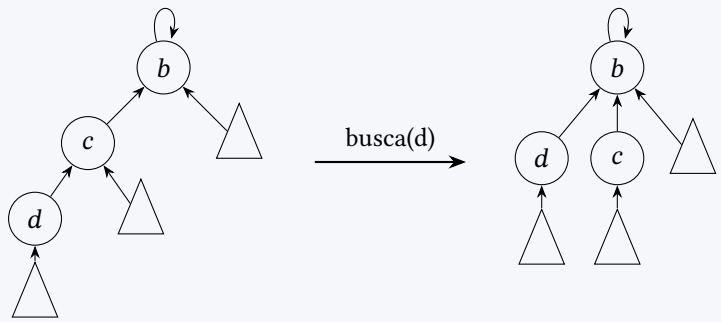
A Proposição 4.5, enunciada para as duas heurísticas ("usando união por tamanho ou por altura, cada árvore tem pelo menos 2^h nós"), também vale para a união por altura, com a seguinte prova para o passo de indução. Sejam A e B as árvores unidas, com $h_A \geq h_B$ (a raiz de B é pendurada na de A). Se $h_A > h_B$, a nova árvore tem altura $h' = h_A$ e $t' \geq t_A \geq 2^{h_A} = 2^{h'}$. Se $h_A = h_B$, então $h' = h_A + 1$ e $t' = t_A + t_B \geq 2 \cdot 2^{h_A} = 2^{h'}$. O corolário segue como antes.

Observação 4.2

Comparando as cotas $O(m + n \log n)$ (listas ligadas) e $O(m \log n + n)$ (árvores), ambas com união por tamanho, as árvores *não* são melhores, e podem ser pior. Com $n = 2^k$ elementos, façam-se $n - 1$ uniões de árvores de mesma altura, obtendo uma árvore de altura $\log_2 n$, e depois $m - 2n$ buscas sobre um nó de profundidade $\log_2 n$; o custo é $\Omega(m \log n)$ para $m \geq 4n$, enquanto as listas gastam $O(m + n \log n)$. A vantagem das árvores vem da heurística de *compressão de caminhos*, vista a seguir.

Busca com compressão de caminhos e união por rank: *quando fazemos um* `busca`(x) *aproveitamos o percurso de x até a raiz da árvore e desviamos os ponteiros* `pai`$[y]$*, de todo y nesse caminho, para a raiz; na união penduramos a árvore de menor rank na raiz da de maior rank.*

Exemplo 4.1: Esquema de uma busca com compressão de caminhos



Com compressão de caminhos a altura de uma árvore pode diminuir, mas atualizá-la a cada busca seria caro. Mantemos então, em cada nó, rank$[x]$, iniciado em 0 e alterado apenas por `link`; ele é uma cota superior da altura da subárvore de x (e igual a ela quando não houve compressão). Na união, a raiz de menor rank é pendurada na de maior rank, e, em caso de empate, o rank da raiz que recebe aumenta de 1. A compressão não muda o conjunto de nós de uma árvore nem o rank das raízes, logo a Proposição 4.5 vale com *rank* no lugar de altura (mesma prova da união por altura): toda árvore de raiz r tem pelo menos $2^{\text{rank}[r]}$ nós e, portanto, rank $\leq \lfloor \log_2 n \rfloor$.

O algoritmo para `busca` com compressão de caminhos fica

Algoritmo 21: cria(x)

```
1 pai[x] ← x;
2 rank[x] ← 0;
```

Algoritmo 22: uniao(x, y)

```
1 r_x ← busca(x);
2 r_y ← busca(y);
3 se rank[r_x] ≥ rank[r_y] então link(r_x, r_y);
4 senão link(r_y, r_x);
```

Algoritmo 23: busca(x) com compressão

```
1 se x ≠ pai[x] então pai[x] ← busca(pai[x]);
2 retorne pai[x]
```

Algoritmo 24: link(x, y)

```
1 se rank[x] = rank[y] então rank[x] ← rank[x] + 1;
2 pai[y] ← x;
```

Essas duas heurísticas juntas, a união por *rank* e a compressão de caminhos é bastante eficiente. O próximo resultado, sobre a complexidade de m operações num universo de tamanho n, envolve uma função $\alpha(m, n)$ que cresce muito vagarosamente; na prática podemos

assumir que $\alpha = 4$. Não demonstraremos esse teorema (veja Cormen et al. (2012)), o resultado que mostraremos a seguir é um pouco menos preciso.

Teorema 4.6: (Tarjan, 1975)

Com união por rank e compressão de caminhos, o tempo gasto com m operações num universo de tamanho n, com $m \geq n$ (as n primeiras sendo `cria`), é $O(m\,\alpha(m,n))$.

(Demonstração em Cormen et al. (2012).)

Observação 4.3

A função de Ackermann (na versão de Cormen et al. (2012)) é dada, para $i, j \geq 1$, por

$$A(i,j) = \begin{cases} 2^j & \text{se } i = 1, \\ A(i-1,2) & \text{se } i \geq 2 \text{ e } j = 1, \\ A\big(i-1, A(i,j-1)\big) & \text{se } i \geq 2 \text{ e } j \geq 2, \end{cases}$$

e cresce muito rapidamente: $A(2,j)$ é uma torre de exponenciais com $j+1$ dois, $A(3,1) = 16$ e $A(4,1) = A(2,16)$ é uma torre de 17 dois, imensamente maior que 10^{80} (estimativa do número de átomos do universo observável). Para $m \geq n$ define-se

$$\alpha(m,n) = \min\big\{i \geq 1 : A(i, \lfloor m/n \rfloor) > \lg n\big\},$$

onde $\lg = \log_2$. Como A é crescente em j, $\alpha(m,n) \leq \alpha(n,n)$, e portanto $\alpha(m,n) \leq 4$ para todo $n < 2^{A(4,1)}$. Por exemplo,

n	$\alpha(n,n)$
1 – 3	1
4 – 15	2
16 – 65535	3
65536 – $2^{A(4,1)} - 1$	4

Considere uma sequência de m operações, das quais as n primeiras são `cria`, com união por *rank* e compressão de caminhos. Sejam $b_0 = 1$ e $b_g = 2^{b_{g-1}}$ (isto é, $1, 2, 4, 16, 65536, \ldots$). Particionamos os elementos de $\mathcal{U}$ em *grupos* segundo o *rank* final: o Grupo 0 tem os de *rank* 0 e 1 e, para $g \geq 1$, o Grupo g tem os de *rank* em $(b_{g-1}, b_g] = (K, 2^K]$ com $K = b_{g-1}$. O *rank* satisfaz:

(i) o *rank* de um nó só muda em `link`, quando ele é a raiz que recebe a outra; logo, depois que deixa de ser raiz, não muda mais;

(ii) rank[x] $<$ rank[pai[x]] para todo x que não é raiz (vale na criação; em `link`, se os ranks são iguais o da raiz que re-

cebe aumenta; e a compressão aponta x para um ancestral, de rank maior). Em particular, $\mathrm{rank}[\mathrm{pai}[x]]$ só cresce ao longo do tempo;

(iii) para cada x, seja $D(x)$ a árvore de x no instante em que x deixa de ser raiz (ou no fim, se isso nunca ocorre). Então $|D(x)| \geq 2^{\mathrm{rank}[x]}$ (Proposição 4.5 com *rank*); e se $x \neq y$ têm o mesmo *rank* final, então $D(x) \cap D(y) = \varnothing$;

(iv) para todo k, há no máximo $n/2^k$ nós de *rank* maior que k; em particular, o Grupo g tem no máximo $n/2^K$ nós;

(v) há no máximo $\lg^* n$ grupos.

Justificativas. (iii) A árvore de x no instante em que deixa de ser raiz tem pelo menos $2^{\mathrm{rank}[x]}$ nós, e o seu *rank* já é o final por (i). Suponha $z \in D(x) \cap D(y)$, com $x \neq y$ do mesmo rank r e, sem perda de generalidade, x deixando de ser raiz antes de y (ou y sendo raiz no fim). Nesse instante x é ligado a uma raiz R de rank $\geq r$, e após o `link` a raiz da árvore de z passa a ter rank $> r$ (se $\mathrm{rank}[R] = r$, ele aumenta), e esse rank só cresce. Quando y deixa de ser raiz, y é a raiz da árvore de z com rank r, uma contradição. (iv) Para cada r, os $D(x)$ de rank r são disjuntos e têm $\geq 2^r$ elementos, logo há $\leq n/2^r$ nós de rank r; some sobre $r > k$. (v) O rank é no máximo $\lfloor \lg n \rfloor$ (Proposição 4.5). Se g_0 é o menor g com $b_g \geq \lfloor \lg n \rfloor$, há $g_0 + 1$ grupos, e para $g_0 \geq 1$ vale $b_{g_0-1} < \lg n$, isto é, $n > b_{g_0}$; aplicando lg repetidamente, $\lg^{(g_0)} n > b_0 = 1$, logo $\lg^* n \geq g_0 + 1$.

O custo total é $O(m)$ (cada `cria` e cada `link` custa $O(1)$) mais o custo das $f \leq 2m$ buscas, e o custo de uma busca é o número de nós do caminho $x = y_0, \ldots, y_k$ até a raiz, cujos apontadores são redirecionados. Classificamos os nós y do caminho em dois tipos:

1. Tipo 1: y é a raiz ou filho da raiz, ou $\mathrm{pai}[y]$ está em grupo diferente do de y;
2. Tipo 2: os demais, isto é, y não é a raiz nem filho da raiz e $\mathrm{pai}[y]$ está no mesmo grupo que y.

Tipo 1. Ao longo do caminho o rank cresce estritamente, logo o grupo não decresce e muda no máximo $\lg^* n - 1$ vezes; com a raiz e o filho da raiz, cada busca tem no máximo $\lg^* n + 1$ nós do Tipo 1. Em f buscas são $O(m \lg^* n)$.

Tipo 2. Seja y no Grupo g, com ranks em $(K, 2^K]$. Cada vez que y é do Tipo 2 em uma busca, $\mathrm{pai}[y]$ passa a ser a raiz, cujo rank é maior que o do pai anterior (pois este não era a raiz); como o rank do pai só cresce e o de y é fixo por (i), o rank de $\mathrm{pai}[y]$ assume valores distintos em $(\mathrm{rank}[y], 2^K]$ a cada ocorrência, e depois que sai do grupo y nunca mais é do Tipo 2. Logo y é do Tipo 2 no máximo 2^K vezes. Por (iv) há no máximo $n/2^K$ tais nós, e o Grupo g contribui com no máximo

$$\frac{n}{2^K} \cdot 2^K = n$$

e, somando sobre os $\leq \lg^* n$ grupos, $O(n \lg^* n)$.

Teorema 4.7

Com união por *rank* e compressão de caminhos, o tempo gasto com m operações `cria`/`busca`/`uniao` num universo com n elementos, das quais as n primeiras são `cria`, é $O((m + n) \lg^* n)$. □

Exercícios

Exercício 75. Seja $G = (V, E, \rho)$ um grafo conexo, $U \subsetneq V$ não-vazio e e uma aresta de peso mínimo em $\nabla_G(U)$. Mostre que e pertence a alguma árvore geradora mínima de G.

Exercício 76. Seja $G = (V, E, \rho)$ um grafo conexo e seja C um circuito de G. Mostre que se e é uma aresta de peso máximo entre as arestas de C, então $G - e$ é conexo e existe uma árvore geradora mínima de $G - e$ que também é uma árvore geradora mínima de G. Em particular, isso vale para toda aresta de peso máximo em G que pertence a algum circuito.

Exercício 77. Seja $G = (V, E, \rho)$ um grafo conexo e seja $U \subsetneq V$ não-vazio. Mostre que se e é a *única* aresta de menor custo em $\nabla_G(U)$, então e pertence a toda árvore geradora mínima de G. Mostre, com um exemplo, que a hipótese de unicidade não pode ser omitida.

Exercício 78. Seja $G = (V, E, \rho)$ um grafo conexo e seja $T \subset G$ uma árvore geradora. Mostre que T é uma árvore geradora mínima se e somente se, para toda $e \in E(G) \setminus E(T)$, toda aresta do único circuito C de $T + e$ tem custo no máximo $\rho(e)$.

Exercício 79. Seja $G = (V, E, \rho)$ um grafo conexo. Mostre que se, para todo corte $\nabla_G(U)$ com $U \subsetneq V$ não-vazio, existe uma única aresta de custo mínimo no corte, então G tem exatamente uma árvore geradora mínima. A recíproca dessa afirmação vale? Justifique.

Exercício 80. Seja $G = (V, E, \rho)$ um grafo conexo com pesos positivos nas arestas. Para toda árvore geradora mínima T de G e todo caminho $P \subset T$, o caminho P é um caminho de custo mínimo em G entre as suas extremidades? Justifique.

Exercício 81. Mostre que para toda árvore geradora mínima T de G existe uma ordenação $e_1, \ldots, e_{|E|}$ das arestas de G, com $\rho(e_1) \leq \cdots \leq \rho(e_{|E|})$, tal que o algoritmo de Kruskal, examinando as arestas nessa ordem, devolve T.

Exercício 82. Seja $G = (V, E, \rho)$ um grafo conexo com pesos positivos nas arestas. Mostre que todo conjunto $F \subseteq E$ tal que (V, F) é conexo e $\rho(F) = \sum_{e \in F} \rho(e)$ é mínimo entre os conjuntos com essa propriedade induz uma árvore, isto é, (V, F) é uma árvore. Dê um exemplo com pesos não-positivos em que essa conclusão não vale.

Exercício 83. Considere a seguinte estratégia genérica para computar uma árvore geradora de custo mínimo (AGM). Seja $G = (V, E, \rho)$ um grafo conexo com pesos nas arestas. Dado $S \subseteq E$, dizemos que uma aresta $e \in E \setminus S$ é **boa para** S se $S \cup \{e\} \subseteq E(T)$ para alguma AGM T de G.

Algoritmo 25: AGM(G)

Dado : um grafo conexo $G = (V, E, \rho)$.
Devolve : árvore geradora de custo mínimo.

```
1 S ← ∅;
2 enquanto (V, S) não é árvore geradora faça
3     escolha uma aresta f ∈ E \ S boa para S;
4     S ← S ∪ {f};
5 retorne (V, S)
```

1. Suponha que, em cada iteração do **enquanto**, existe uma aresta boa para S e que um *oráculo* a entrega ao algoritmo na linha 3. Prove que o laço termina e que AGM(G) devolve uma AGM de G. (Dica: determine e prove um invariante para o laço.)
2. Sejam T uma AGM de G, $S \subseteq E(T)$ e $U \subsetneq V$ não-vazio tal que $\nabla_G(U) \cap S = \emptyset$. Prove que toda aresta e de custo mínimo em $\nabla_G(U)$ é boa para S. (Dica: considere os casos $e \in E(T)$ e $e \notin E(T)$; no segundo, note que a aresta que se remove de T pertence ao corte e, portanto, não pertence a S.)
3. Conclua que, se $S \subseteq E(T)$ para alguma AGM T de G e (V, S) não é árvore geradora, então existe uma aresta boa para S; logo a hipótese do item 1 sempre se verifica. (Dica: tome U como o conjunto de vértices de uma componente de (V, S).)
4. Use o item 2 para provar a correção dos algoritmos de Dijkstra–Jarník–Prim e Kruskal: mostre que, em cada iteração, a aresta inserida em S é boa para o S corrente (suponha, por indução, que o S corrente está contido em alguma AGM). (Dica: no Prim use o corte $\nabla_G(U)$ da iteração; no Kruskal, use o corte definido pela componente de uma das pontas da aresta em (V, S) e a ordem em que as arestas são examinadas.)